Do We Need More Bikes?
Project in Statistical Machine Learning

Eric Björling

William Degrer

Elisa Marie Hurt

Upali Bera

Abstract

The project focuses on the management of the 24/7 public bicycle sharing system, Capital Bikeshare located in Washington D.C. We address the binary classification problem of predicting increased bike demand for the Department of Transportation for better mobility and sustainability. Using a dataset of 1,600 observations, we analyzed temporal and meteorological features to identify when stock increases are needed. We explored several classification methods, including Logistic Regression, Linear and Quadratic Discriminant Analysis, K-Nearest Neighbors, Random Forest, and Gradient Boosting. Gradient Boosting was selected as our production model which achieved a weighted F1-score of 0.92.

Number of group members: 4

1 Introduction

The Capital Bikeshare system is a cornerstone in the Washington D.C transit network. It offers a sustainable approach compared to using cars for travel. However, the system struggles with a major logistical bottleneck: identifying exactly when and where to move bikes. In this project, we aim to help the District Department of Transportation [2], by deploying a ML model that predicts whether the stock of bikes needs to be increased (`high_bike_demand`) or not (`low_bike_demand`) at specific hours. Correctly identifying these high demand periods is essential for the department to ensure and maintain bike availability, and by extension also support the city's environmental goals.

2 Exploratory Data Analysis

Feature Exploration:
We analyzed the provided dataset of 1600 samples to better our understanding of the underlying distributions and relationships before modeling. The raw dataset contained 15 potential input features. After preprocessing, we utilized 14 features to predict the binary target variable `increase_stock`.

- We identified 8 **numerical features** representing meteorological conditions: `temp`, `dew`, `humidity`, `precip`, `snowdepth`, `windspeed`, `cloudcover`, and `visibility`.
- The feature `snow` (falling snow) was excluded from the model input as we found it to be constant (zero variance) across the training set. However, `snowdepth` was kept as it contained valid non-zero values.

- We identified 6 **categorical features**. These include the ordinal/cyclical features `hour_of_day`, `day_of_week`, and `month`. These act as proxies for periodic patterns and commuter behavior. The dataset also includes three binary indicator features: `holiday`, `weekday`, and `summertime`.

Class Imbalance:
An important finding was the significant class imbalance. "Low Demand" instances outnumber "High Demand" instances by a ratio of approximately 4.5:1. This imbalance is why we decided to use the Weighted F1-Score as our evaluation metric. Accuracy is not a good metric here because a naive model could just guess 'Low Demand' every time and still get a high score.

Correlations and Trends:
Visual inspection confirmed that demand is strongly correlated with `hour_of_day` and `temp`. Our analysis of the hourly trends revealed a clear commuting pattern where demand increases from morning through afternoon. As shown in Figure 1.

- The highest demand occurs at 17:00–18:00 with a 60–75% high demand rate.
- There is a smaller peak around 15:00–16:00.
- Late night and early morning hours (00:00–07:00) show nearly no high demand

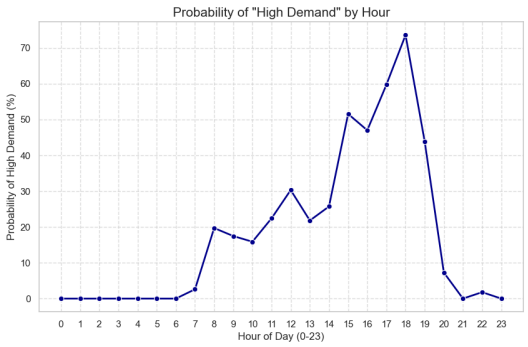

Figure 1: Average bike demand by hour.

The weekly data shows several clear trends. As shown in Figure 2.

- Highest demand happens on Day 5 (Friday) and Day 6 (Saturday).
- Lowest demand happens days 2–3 (Tuesday-Wednesday) which show the lowest probability of high demand.

When looking at the monthly data, a seasonal pattern is identified with warmer weather indicating higher demand as shown in Figure 3.

- The highest demand happens in months 4, 6, and 9 (April, June, and September) with a 29–30% high demand rate.
- Winter months (1, 2, 12) have significantly lower demand (4–12%).

Our analysis reveals behavioral differences between working days and non-working days. As can be seen from Figure 4.

- **Weekends (Non-working days):** Show the highest demand with a 25% high demand rate. This is significantly higher than regular weekdays.
- **Regular Weekdays:** Show a lower high demand rate of approximately 15%.
- **Holidays:** Show an intermediate demand level of 17%. While slightly higher compared to regular workdays, holidays do not reach the peak demand levels that happens on weekends. There is not a noticeable difference between holidays and non-holidays(18%).

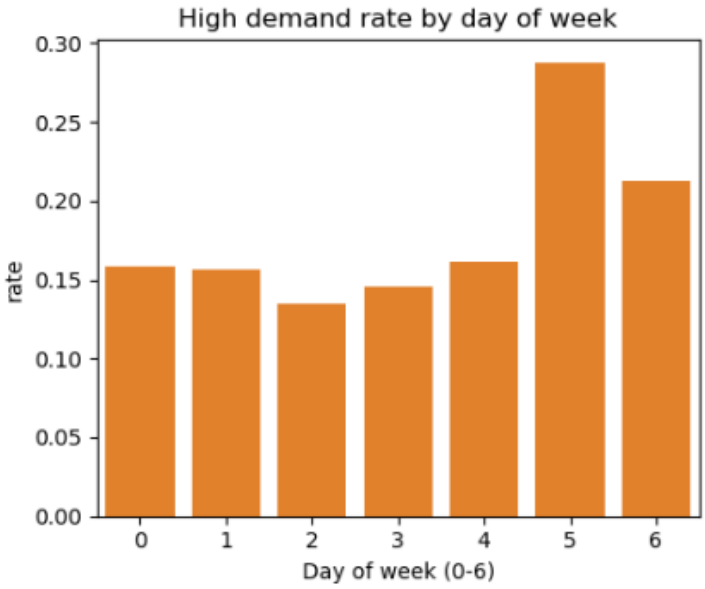

Figure 2: Average bike demand by weekday.

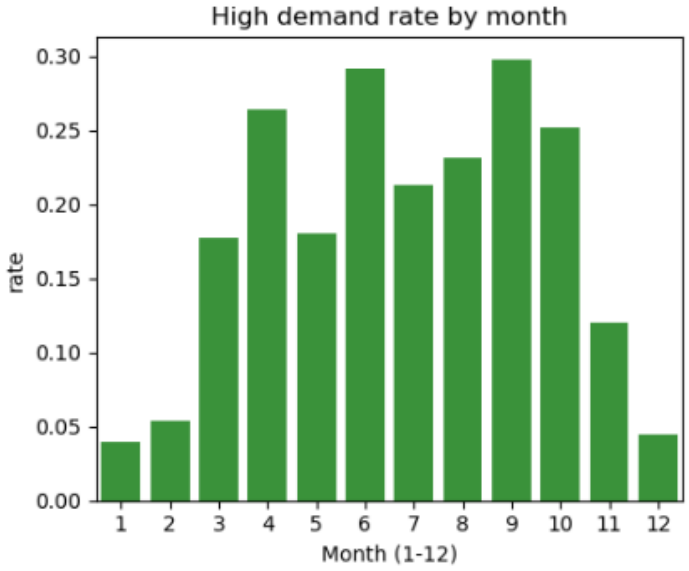

Figure 3: Average bike demand by month.

Weekends generate the highest demand, likely due to non-commute activities. Regular weekdays show a lower commuting pattern. The sample size for holidays is small. However, the data shows this category generally requires a stock increase more frequently than average tuesdays.

Weather conditions and their correlation with bike demand can be seen from Figure 5.

- **Precipitation (Rain/Snow):** Rain acts is in a slight negative correlation with demand (-0.059).
- **Temperature:** There is a strong positive correlation (r = +0.337) between temperature and demand.
- **Humidity:** We observed a strong negative correlation (r = -0.309).

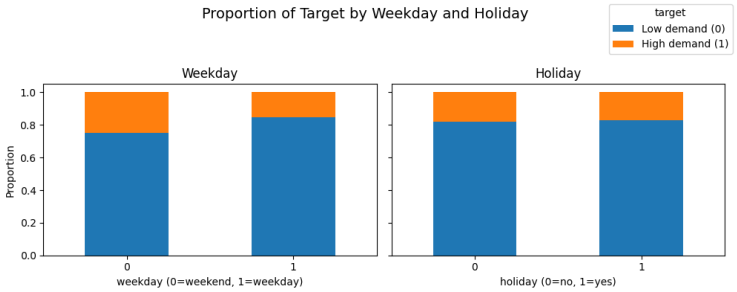

Figure 4: Average bike demand by weekday and holiday.

Temperature and humidity have a almost linear relationship with demand (positive and negative respectively). Precipitation acts as a binary "switch" that shuts down demand.

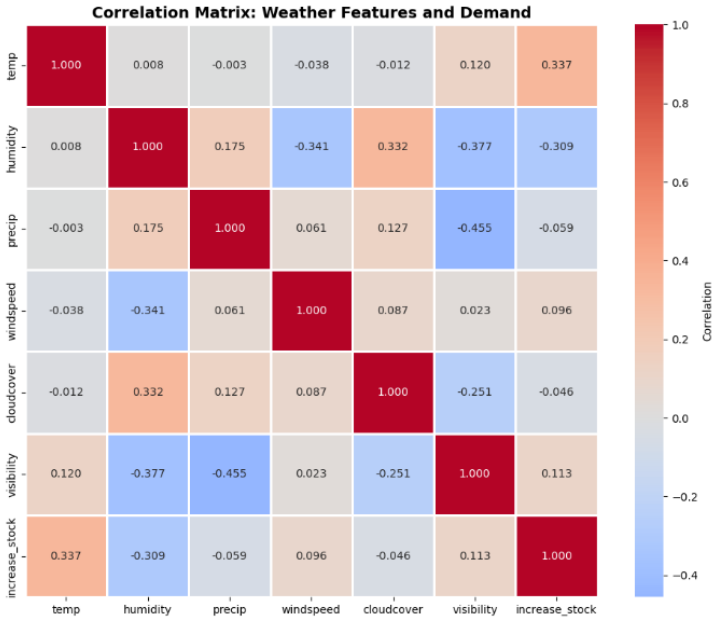

Figure 5: Weather features and demand correlation matrix.

3 Model Development

We implemented the following methods from the classification families, as described in course literature [4][3]: Logistic Regression, Linear and Quadratic Discriminant Analysis for probabilistic classification, K-Nearest Neighbors (KNN), Random Forest, and Gradient Boosting. We also implemented a naive benchmark using Scikit-learn [5]. All models were tuned using cross-validation to prevent overfitting.

3.1 Mathematical Descriptions and Methodology

To address the binary classification of `increase_stock`, we implemented and evaluated several models from five distinct families. Each method was chosen to capture different aspects of the relationship between meteorological conditions and bicycle demand.

Naive Benchmark: To establish a baseline for performance, we defined a naive classifier $C_0(x)$ that ignores input features and consistently predicts the most frequent class (`low_bike_demand`) from

the training set:

$$\hat{y} = \text{mode}(\{y_1, ..., y_n\}) \tag{1}$$

This benchmark is essential for evaluating the actual predictive gain of more complex models, especially given the observed 4.5:1 class imbalance in our dataset.

Logistic Regression: This model estimates the probability of the positive class using the sigmoid function:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta^T x)}}, \tag{2}$$

where β are coefficients learned by maximizing the likelihood of the training data. We chose this method for its high interpretability, as the coefficients directly show how features like `temp` influence demand. To ensure stable coefficients, all numerical features were scaled using `StandardScaler`.

Linear and Quadratic Discriminant Analysis: LDA assumes that each class follows a Gaussian distribution with a common covariance matrix Σ. The discriminant function is:

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k \tag{3}$$

QDA relaxes this by allowing each class its own covariance matrix Σ_k, which results in a quadratic decision boundary:

$$\delta_k(x) = -\frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) + \log \pi_k \tag{4}$$

LDA was implemented as a stable linear baseline, while QDA was motivated by its ability to capture non-linear interactions between weather variables. We used a regularization parameter (`reg_param=0.0001`) in QDA to manage potential multicollinearity between features.

K-Nearest Neighbors (KNN): KNN is a non-parametric method that classifies a new observation by identifying the k nearest training samples using the Euclidean distance metric:

$$d(x, x_i) = \sqrt{\sum_{j=1}^{p} (x_j - x_{ij})^2} \tag{5}$$

The advantage of KNN is its flexibility in capturing local demand patterns. In our application, we tuned $k = 10$ with distance-weighting to smooth out noise in the meteorological data.

Random Forest: This ensemble method constructs B decorrelated trees using bootstrap aggregating (bagging). For each tree T_b, a random subset of m features is used at each split to reduce correlation. The final prediction is a majority vote:

$$\hat{y} = \text{mode}\{T_1(x), ..., T_B(x)\} \tag{6}$$

Random Forest was chosen for its robustness against overfitting and its inherent ability to handle interactions between categorical proxies and numerical weather data.

Gradient Boosting: Gradient Boosting [1] is an iterative ensemble technique. In each stage m, a new tree $h_m(x)$ is fit to the negative gradient of the loss function (pseudo-residuals) of the previous prediction $F_{m-1}(x)$. The model is updated using a learning rate ν:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x) \tag{7}$$

The main advantage of this method is its superior predictive power for complex datasets. We applied it as our final production model, utilizing `scale_pos_weight` to specifically address the imbalance between low and high demand periods.

3.2 Application and Tuning

We preprocessed the data by One-Hot Encoding categorical variables and scaling numerical features using `StandardScaler` for the distance-based and linear models (Logistic Regression, LDA, QDA, KNN) to ensure features with larger scales did not dominate the distance metrics. We utilized Grid Search for hyperparameter tuning, focusing on parameters that control model complexity and prevent overfitting:

- **Logistic Regression:** We tuned the inverse regularization strength $C \in \{0.01, 0.1, 1, 10, 100\}$. $C = 1$ was selected to provide a balance between fitting the training data and maintaining a simple model.
- **LDA:** Standard LDA is parameter-free (estimating means/covariance directly). No explicit hyperparameter tuning was applied.
- **QDA:** We tuned the regularization parameter (`reg_param`) to handle potential collinearity.
- **KNN:** We tuned $k \in \{1, 3, 5, 7, 10, 15, 20\}$ and weight functions. $k = 10$ with uniform weights was chosen as it effectively smoothed local noise in the dataset.
- **Random Forest:** We tuned the model using grid search, optimizing for `n_estimators` and `max_depth`.
- **Gradient Boosting (XGBoost):** Due to its sensitivity, we performed an extensive grid search over `learning_rate`, `max_depth`, and `n_estimators`. The search aimed to find a low learning rate paired with a sufficient number of trees to minimize the log-loss while handling class imbalance via `scale_pos_weight`.

3.3 Experimental Evaluation

We evaluated models using Weighted F1-Score due to the class imbalance.

Table 1: Model comparison based on Weighted F1-Score. The F1 metric was chosen to account for the class imbalance (4.5:1).

Model Family	Weighted F1-Score	Accuracy	Macro recall	Performance Notes
Naive Benchmark	0.7372	0.8187	0.5000	Baseline (Majority Class)
Logistic Regression	0.8188	0.8000	0.8175	Linear model with regularization tuning
LDA	0.8423	0.8562	0.6840	Strong linear baseline
QDA	0.6498	0.6062	0.7058	High recall but lower precision
K-Nearest Neighbors	0.8016	0.8281	0.6064	Sensitive to high dimensionality
Random Forest	0.9071	0.9062	0.8219	Robust, near-top performance
Gradient Boosting	**0.9192**	**0.9219**	**0.8382**	**Best Model (Selected)**

Performance Analysis: The results in Table 1 highlight the strengths of both linear and ensemble methods.

- **Linear Models:** Both Logistic Regression (F1=0.8188) and LDA (F1=0.8423) performed well, suggesting that a significant portion of the decision boundary can be captured by linear methods. LDA slightly outperformed Logistic Regression, likely due to its assumption of Gaussian-distributed classes.
- **Quadratic Models:** QDA achieved F1 = 0.6498. While QDA typically aims for high recall, the lower overall performance here suggests potential overfitting or violations of the Gaussian assumption. To address potential multicollinearity (e.g., temp vs dew, $r = 0.87$), we investigated whether removing the highly correlated dew feature would improve performance. We found that removing dew decreased performance: F1 declined from 0.6498 to 0.6021. These results indicate that despite its high correlation with temperature, dew retains independent predictive value. The regularization parameter (reg_param $= 0.0001$) effectively manages multicollinearity without requiring feature removal.
- **Success of Ensembles:** Gradient Boosting achieved the highest F1-score (0.9192). The Feature Importance plot (Figure 7) identifies **hour_of_day** and **visibility** as dominant drivers. This suggests the model leverages the non-linear interaction between specific times of day and favorable weather conditions.

While we compared all models on weighted F1-Score, our selected Gradient Boosting model also achieved a global accuracy of 92% and a weighted precision of 92% which confirms it is not just guessing the majority class.

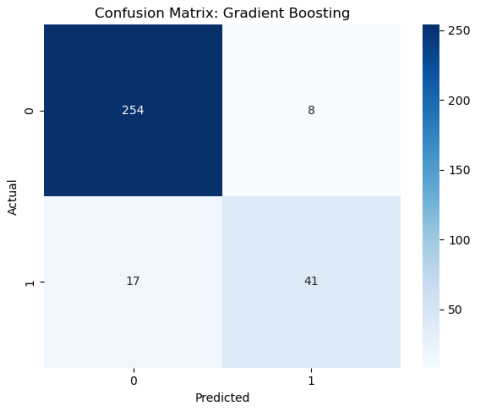

Figure 6: Confusion Matrix (Gradient Boosting). The model demonstrates a balanced capability to identify high demand periods.

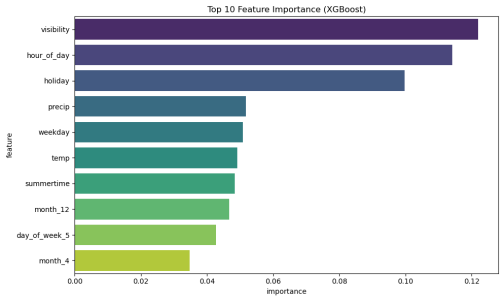

Figure 7: Top 10 Feature Importance (XGBoost). Visibility and hour of day are the strongest predictors.

3.4 Model Selection

We selected **Gradient Boosting** as our production model because it achieved the highest Weighted F1-Score (0.9192). This ensemble method was motivated by its inherent ability to capture non-linear interactions between temporal variables and weather conditions which linear models partially missed. Furthermore it provided better stability against the multicollinearity issues that hindered the QDA model.

4 Conclusion

We successfully developed a machine learning pipeline to predict bike sharing demand in Washington, D.C. Our analysis demonstrated that demand is driven by both linear and non-linear interactions between temporal features (hour of day, season) and weather conditions (temperature, humidity).

Linear models - specifically Logistic Regression and LDA - provided good baselines and captured a substantial portion of the underlying patterns. However, ensemble methods, particularly Gradient Boosting, achieved superior performance with a weighted F1-score of 0.9192, indicating that non-linear relationships and feature interactions are important for accurate demand prediction.

Acknowledgments and Disclosure of Funding

We acknowledge the course instructors for the guidance and the provided dataset [2].

References

[1] T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016.

[2] Department of Information Technology. Statistical machine learning course material and dataset, 2025. Course Code: 1RT700.

[3] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, 2nd edition, 2009.

[4] A. Lindholm, N. Wahlström, F. Lindsten, and T. B. Schön. *Machine Learning - A First Course for Engineers and Scientists*. Cambridge University Press, 2022.

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot,

and E. Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

A Appendix: Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Filter warnings
import warnings
warnings.filterwarnings("ignore")

# Model imports
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# Selection and metrics
from sklearn.model_selection import train_test_split, GridSearchCV,
    RandomizedSearchCV, cross_validate
from sklearn.metrics import classification_report, confusion_matrix,
    f1_score, accuracy_score, precision_score, recall_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# 1. Data Preparation ============================================
# Load Training Data
df = pd.read_csv("training_data_ht2025.csv")

# Define feature groups
cat_features = ['hour_of_day', 'day_of_week', 'month', 'holiday', '
    weekday', 'summertime']
num_features = ['temp', 'dew', 'humidity', 'precip', 'snowdepth', '
    windspeed', 'cloudcover', 'visibility']

# Map target to binary
df['target'] = df['increase_stock'].map({'low_bike_demand': 0, '
    high_bike_demand': 1})

# 2. Exploratory Data Analysis (EDA) ============================

# Fig 1: Hourly Demand
plt.figure(figsize=(10, 5))
hourly = df.groupby('hour_of_day')['target'].mean() * 100
plt.plot(hourly.index, hourly.values, marker='o', color='navy')
plt.title('Probability of "High Demand" by Hour')
plt.xlabel('Hour of Day')
plt.ylabel('Probability (%)')
plt.xticks(range(0, 24))
plt.grid(True)
plt.savefig('fig1_hourly_demand.png') # Saved for report
plt.show()

# Fig 2: Weekly Demand
plt.figure(figsize=(8, 5))
weekly = df.groupby('day_of_week')['target'].mean()
```

```
sns.barplot(x=weekly.index, y=weekly.values, color='orange')
plt.title('High demand rate by day of week')
plt.xlabel('Day of Week (0-6)')
plt.savefig('fig2_weekly_demand.png') # Saved for report
plt.show()

# Fig 3: Monthly Demand
plt.figure(figsize=(8, 5))
monthly = df.groupby('month')['target'].mean()
sns.barplot(x=monthly.index, y=monthly.values, color='forestgreen')
plt.title('High demand rate by month')
plt.xlabel('Month (1-12)')
plt.savefig('fig3_monthly_demand.png') # Saved for report
plt.show()

# Fig 4: Weekday vs Holiday
fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)

wd_prop = (
        df.groupby('weekday')['target']
              .value_counts(normalize=True)
                    .unstack(fill_value=0)
)
wd_prop.plot(kind='bar', stacked=True, ax=axes[0], legend=False)
axes[0].set_title('Weekday')
axes[0].set_xlabel('weekday (0=weekend, 1=weekday)')
axes[0].set_ylabel('Proportion')
axes[0].tick_params(axis='x', labelrotation=0)

hol_prop = (
        df.groupby('holiday')['target']
              .value_counts(normalize=True)
                    .unstack(fill_value=0)
)
hol_prop.plot(kind='bar', stacked=True, ax=axes[1], legend=False)
axes[1].set_title('Holiday')
axes[1].set_xlabel('holiday (0=no, 1=yes)')
axes[1].tick_params(axis='x', labelrotation=0)
handles, _ = axes[0].get_legend_handles_labels()
labels = ['Low demand (0)',
          'High demand (1)']
          fig.legend(
                handles,
                    labels,
                        title='Target',
                            loc='upper right',
                                ncol=1
          )
          fig.suptitle('Proportion of Target by Weekday and Holiday',
    fontsize=14)
          plt.tight_layout(rect=[0, 0, 1, 0.9])
          plt.savefig('fig4_weekday_holiday.png') # Saved for report
          plt.show()

          # Fig 5: Correlation Matrix
          plt.figure(figsize=(10, 8))
          plot_cols = ['temp', 'humidity', 'precip', 'windspeed', '
    cloudcover', 'visibility', 'target']
          corr_data = df[plot_cols].rename(columns={'target': '
    increase_stock'})
          sns.heatmap(corr_data.corr(), annot=True, cmap='coolwarm',
    fmt=".3f", linewidths=1, linecolor='white')
          plt.title('Correlation Matrix', fontweight='bold')
          plt.savefig('fig5_correlation.png') # Saved for report
          plt.show()
```

```
114
115         # 3. Preprocessing ========================================
116
117         # Drop "snow" because of zero variance in training set
118         if 'snow' in df.columns:
119             df = df.drop(columns=['snow'])
120
121             # One-Hot Encoding for categorical proxies
122             cat_cols_to_encode = ['day_of_week', 'month']
123             df_processed = pd.get_dummies(df, columns=
    cat_cols_to_encode, drop_first=True)
124
125             # Define Features and Target
126             X = df_processed.drop(columns=['increase_stock', 'target
    '])
127             y = df_processed['target']
128
129             # 80/20 Train-Validation Split
130             X_train, X_val, y_train, y_val = train_test_split(X, y,
    test_size=0.2, random_state=42, stratify=y)
131
132             # Scaling numerical features
133             scaler = StandardScaler()
134             X_train_scaled = scaler.fit_transform(X_train)
135             X_val_scaled = scaler.transform(X_val)
136
137             print("\n--- Model Benchmarking and Tuning ---")
138
139             # 4. Model Training & Evaluation
    ========================================
140
141             # --- 1. Naive Baseline ---
142             dummy = DummyClassifier(strategy="most_frequent")
143             dummy.fit(X_train, y_train)
144             val_f1_naive = f1_score(y_val, dummy.predict(X_val),
    average='weighted')
145             print(f"Naive Benchmark Weighted F1: {val_f1_naive:.4f}"
    )
146
147             # --- 2. Logistic Regression ---
148             # Pipeline setup
149             log_pipe = Pipeline([
150                   ('scaler', StandardScaler()),
151                       ('clf', LogisticRegression(class_weight='
    balanced', max_iter=1000, random_state=42))
152             ])
153
154             # TUNING (Commented out)
155             # param_grid_log = {'clf__C': [0.01, 0.1, 1, 10, 100]}
156             # grid_log = GridSearchCV(log_pipe, param_grid_log, cv
    =5, scoring='f1_weighted')
157             # grid_log.fit(X_train, y_train)
158             # best_log = grid_log.best_estimator_
159             # print("Best LogReg Params:", grid_log.best_params_)
160
161             # Hardcoded Best Model
162             log_model = LogisticRegression(C=10, class_weight='
    balanced', max_iter=1000, random_state=42)
163             log_model.fit(X_train_scaled, y_train)
164             pred_log = log_model.predict(X_val_scaled)
165
166             print(f"Logistic Regression Weighted F1: {f1_score(y_val
    , pred_log, average='weighted'):.4f}")
167
168             # --- 3. Linear Discriminant Analysis (LDA) ---
```

```
            lda = LinearDiscriminantAnalysis()
            lda.fit(X_train_scaled, y_train)
            pred_lda = lda.predict(X_val_scaled)
            print(f"LDA Weighted F1: {f1_score(y_val, pred_lda,
    average='weighted'):.4f}")

            # --- 4. Quadratic Discriminant Analysis (QDA) ---
            # QDA with all features
            qda = QuadraticDiscriminantAnalysis(reg_param=0.0001)
            qda.fit(X_train_scaled, y_train)
            pred_qda = qda.predict(X_val_scaled)
            print(f"QDA (All Features) Weighted F1: {f1_score(y_val,
     pred_qda, average='weighted'):.4f}")

            # QDA without 'dew'
            dew_col_idx = list(X.columns).index('dew') if 'dew' in X
    .columns else -1
            if dew_col_idx != -1:
                X_train_no_dew = np.delete(X_train_scaled,
    dew_col_idx, axis=1)
                    X_val_no_dew = np.delete(X_val_scaled,
    dew_col_idx, axis=1)

                            qda_no_dew =
    QuadraticDiscriminantAnalysis(reg_param=0.0001)
                                qda_no_dew.fit(X_train_no_dew,
    y_train)
                                    pred_qda_no_dew = qda_no_dew.
    predict(X_val_no_dew)
                                        print(f"QDA (No Dew)
    Weighted F1: {f1_score(y_val, pred_qda_no_dew, average='weighted')
    :.4f}")

                                        # --- 5. K-Nearest Neighbors
     (KNN) ---
                                        # TUNING (Commented out)
                                        # knn_grid = GridSearchCV(
                                          #     KNeighborsClassifier
    (),
                                          #     param_grid={'
    n_neighbors': [3, 5, 10, 15], 'weights': ['uniform', 'distance']},
                                          #     cv=5,
                                          #     scoring='f1_weighted
    '
                                          # )
                                          # knn_grid.fit(
    X_train_scaled, y_train)
                                          # print(f"KNN Best Params:
     {knn_grid.best_params_}")

                                          # Hardcoded Best
                                          knn_best =
    KNeighborsClassifier(n_neighbors=10, weights='distance')
                                          knn_best.fit(
    X_train_scaled, y_train)
                                          pred_knn = knn_best.
    predict(X_val_scaled)
                                          print(f"KNN (Tuned)
    Weighted F1: {f1_score(y_val, pred_knn, average='weighted'):.4f}")

                                          # --- 6. Random Forest ---
                                          # TUNING (Commented out)
                                          # rf_params = {
                                              #     'n_estimators':
    [100, 200, 300],
```

```
                                                #     'max_depth': [
    None, 10, 20],
                                                #     '
    min_samples_split': [2, 5],
                                                #     'class_weight':
    ['balanced', None]
                                                # }
                                                # grid_rf =
    GridSearchCV(RandomForestClassifier(random_state=42), rf_params,
    cv=5, scoring='f1_weighted', n_jobs=-1)
                                                # grid_rf.fit(
    X_train_scaled, y_train)
                                                # print(f"Random
    Forest Best Params: {grid_rf.best_params_}")

                                                # Hardcoded Best
                                                rf_best =
    RandomForestClassifier(n_estimators=200, max_depth=20,
    min_samples_split=5, class_weight='balanced', random_state=42)
                                                rf_best.fit(
    X_train_scaled, y_train)
                                                pred_rf = rf_best.
    predict(X_val_scaled)
                                                print(f"Random Forest
    (Tuned) Weighted F1: {f1_score(y_val, pred_rf, average='weighted')
    :.4f}")


                                                #
    ========================================
                                                # 7. Final Production
    Model: XGBoost
                                                #
    ========================================
                                                print("\n--- Training
    Final Production Model (XGBoost) ---")

                                                # --- TUNING &
    VALIDATION STRATEGY (Commented out) ---
                                                # The specific
    parameters (n_estimators=2396) were derived using Early Stopping.
                                                #
                                                # 1. Grid Search for
    structural params (depth, learning_rate):
                                                # xgb_grid =
    GridSearchCV(
                                                    #
    XGBClassifier(n_estimators=1000, tree_method='hist'),
                                                    #     param_grid =
     {
                                                        #         '
    max_depth': [6, 8, 10],
                                                        #         '
    learning_rate': [0.01, 0.05],
                                                        #         '
    scale_pos_weight': [1.0, 1.03] # Accounting for slight imbalance
                                                    #     },
                                                    #     scoring
    ='f1_weighted', cv=3
                                                # )
                                                # xgb_grid.fit
    (X_train_scaled, y_train)
                                                # best_params
    = xgb_grid.best_params_
                                                #
```

```
                                                    # 2. Determine optimal n_estimators with Early Stopping:
                                                    # model_es = XGBClassifier(**best_params, n_estimators=5000, random_state=42)
                                                    # model_es.fit(X_train_scaled, y_train,
                                                    #              eval_set=[(X_val_scaled, y_val)],
                                                    #              early_stopping_rounds=50, verbose=False)
                                                    # print(f"Optimal n_estimators: {model_es.best_iteration + 1}")
                                                    # # Result -> 2396 trees

                                                    # Exact parameters derived from the strategy above
xgb_production_params = {
                                                        "n_estimators": 2396,          # Found via early stopping
                                                        "learning_rate": 0.01,
  "max_depth": 8,
      "scale_pos_weight": 1.03,      # Tuned for class balance
          "eval_metric": "logloss",
              "random_state": 42,
                  "tree_method": "hist"
                                                    }

                                                    final_model = XGBClassifier(**xgb_production_params)
                                                    final_model.fit(X_train_scaled, y_train)

                                                    # Evaluation
                                                    y_pred_final = final_model.predict(X_val_scaled)
                                                    print("\n--- Final Production Model Report (XGBoost) ---")
                                                    print(classification_report(y_val, y_pred_final))
                                                    print(f"Final Weighted F1 Score: {f1_score(y_val, y_pred_final, average='weighted'):.4f}")

                                                    # --- Visualizations ---

                                                    # 1. Confusion Matrix
                                                    plt.figure(figsize=(6, 5))
                                                    sns.heatmap(confusion_matrix(y_val, y_pred_final), annot=True, fmt='d', cmap='Blues')
                                                    plt.title("Confusion Matrix: Gradient Boosting")
                                                    plt.xlabel("Predicted")
```

```
                                                    plt.ylabel("Actual")
                                                    plt.tight_layout()
                                                    plt.savefig('confusion_matrix_xgb.png')
                                                    print("Confusion Matrix saved to 'confusion_matrix_xgb.png'")

                                                    # 2. Top Features Importance
                                                    importances = final_model.feature_importances_
                                                    feature_names = list(X.columns)
                                                    feat_df = pd.DataFrame({'feature': feature_names, 'importance': importances})
                                                    feat_df = feat_df.sort_values(by='importance', ascending=False).head(10)

                                                    plt.figure(figsize=(10, 6))
                                                    sns.barplot(x='importance', y='feature', data=feat_df, palette='viridis')
                                                    plt.title("Top 10 Feature Importance (XGBoost)")
                                                    plt.tight_layout()
                                                    plt.savefig('feature_importance_xgb.png')
                                                    print("Feature Importance saved to 'feature_importance_xgb.png'")

                                                    # 8. Submission File Generation
                                                    TEST_FILE_NAME = "test_data_fall2025.csv"

                                                    try:
                                                        print(f"\n--- Generating Predictions for {TEST_FILE_NAME} ---")
df_test = pd.read_csv(TEST_FILE_NAME)

  # --- Preprocessing Test Set ---
      # 1. Drop snow
          if 'snow' in df_test.columns:
                  df_test = df_test.drop(columns=['snow'])

                          # 2. One-Hot Encode (Align columns with training)
                              df_test_processed = pd.get_dummies(df_test, columns=cat_cols_to_encode, drop_first=True)

                                      # 3. Align columns (Add missing cols with 0, drop extra cols)
```

```
                                                # Get missing columns in
 the test set
                                                    missing_cols = set(X
.columns) - set(df_test_processed.columns)
                                                        for c in
missing_cols:
df_test_processed[c] = 0
                                                                    #
Reorder columns to match training X
df_test_processed = df_test_processed[X.columns]


        # 4. Scale
            X_test_scaled = scaler.transform(df_test_processed)


                    # --- Prediction ---
                        predictions = final_model.predict(
X_test_scaled)


                                # --- Formatting for Submission
---
                                    submission_string = ",".join(
map(str, predictions))


                                            # Save to CSV
                                                with open("
predictions.csv", "w") as f:
                                                            f.write(
submission_string)


  print("Success! 'predictions.csv' has been created.")
      print(f"Prediction count: {len(predictions)}")
```

```
            print(f"Sample: {submission_string[:20]}...")
341
342
            except FileNotFoundError:
343
                print(f"WARNING: Test file '{TEST_FILE_NAME}' not found.")
344
                except Exception as e:
345
                    print(f"Error: {e}")
                                                    })
                                                }
                                            )
                                        }
                                    )
            ])
        )]
)
)
```